

VIRGO

SLM V1.0

Building a 120M-Parameter Transformer Language Model from Scratch

A Self-Learning Engineering Project

“Not Everybody Needs The Blueprint.”

Punit Kumar Kashyap

First-Year Undergraduate, Department of Engineering Physics
Indian Institute of Technology Dharwad

2026

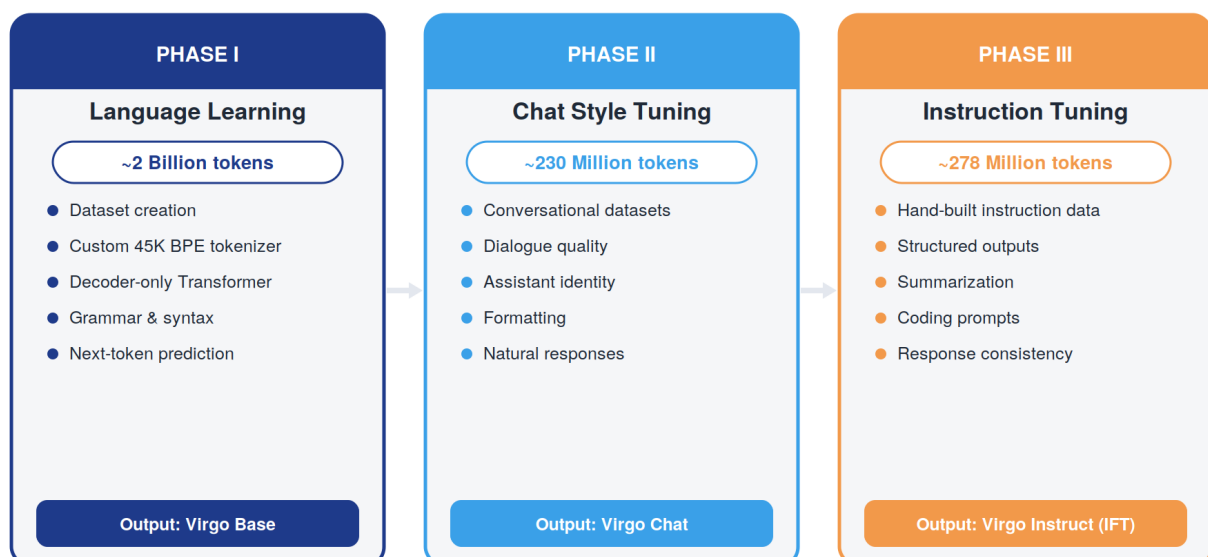
01 Abstract

Virgo SLM V1.0 is a 120-million-parameter decoder-only Transformer language model, independently designed, trained, and fine-tuned by the author as a first-year undergraduate at IIT Dharwad. This report documents the complete engineering process — architecture design, dataset construction, training methodology, and evaluation — behind the model, alongside the practical lessons learned at each stage.

Development proceeded through three defined phases. Phase I established core language modeling capability — grammar, vocabulary, and next-token prediction — over approximately 2 billion pretraining tokens, producing Virgo Base. Phase II adapted the model into a conversational assistant, Virgo Chat, using roughly 230 million tokens of dialogue data. Phase III applied a 278-million-token instruction-tuning dataset, hand-curated by the author over several months, to produce the final model, Virgo Instruct.

STUDENT REFLECTION

Virgo is small by industry standards, and I know it. It isn't a competitor to GPT, Gemini, or Claude — it was never meant to be. To me, it's proof that one student with a laptop, patience, and enough failed training runs can build something that actually works, actually generates language, and teaches more than any textbook could.



02 Why I Started Virgo

This project was motivated by a specific gap: theoretical familiarity with Transformer architectures did not translate into practical understanding of their internal behavior. Reading and reproducing the mathematics of self-attention is not equivalent to being able to explain, with confidence, why a model selects one token over another during generation. Closing that gap required building a complete language model end-to-end rather than studying existing ones.



STUDENT REFLECTION

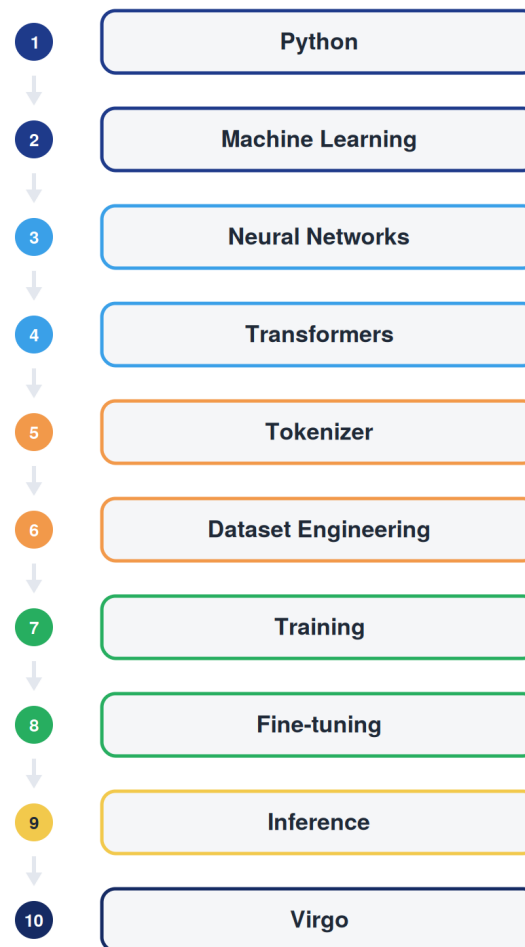
I could recite the equations for self-attention on a whiteboard, but I couldn't picture what was actually happening inside a Transformer when a prompt went in and a sentence came out. That bothered me more than it should have — so I opened a blank Python file and decided to find out for myself.

“Not Everybody Needs The Blueprint.”

The project's guiding principle is that there is no single prescribed path to understanding how language models work. In the absence of a lab position, a mentor, or a structured curriculum for this specific goal, the approach taken here was incremental: a bigram model, then a single attention head, then a full decoder block — each implementation deepening the understanding built by the last.

03 My Learning Journey

Building Virgo required a structured progression of prerequisite knowledge, developed over the course of the first academic year alongside regular coursework in Engineering Physics. The sequence below reflects the order in which each concept was studied and applied, with each stage serving as a direct prerequisite for the next.

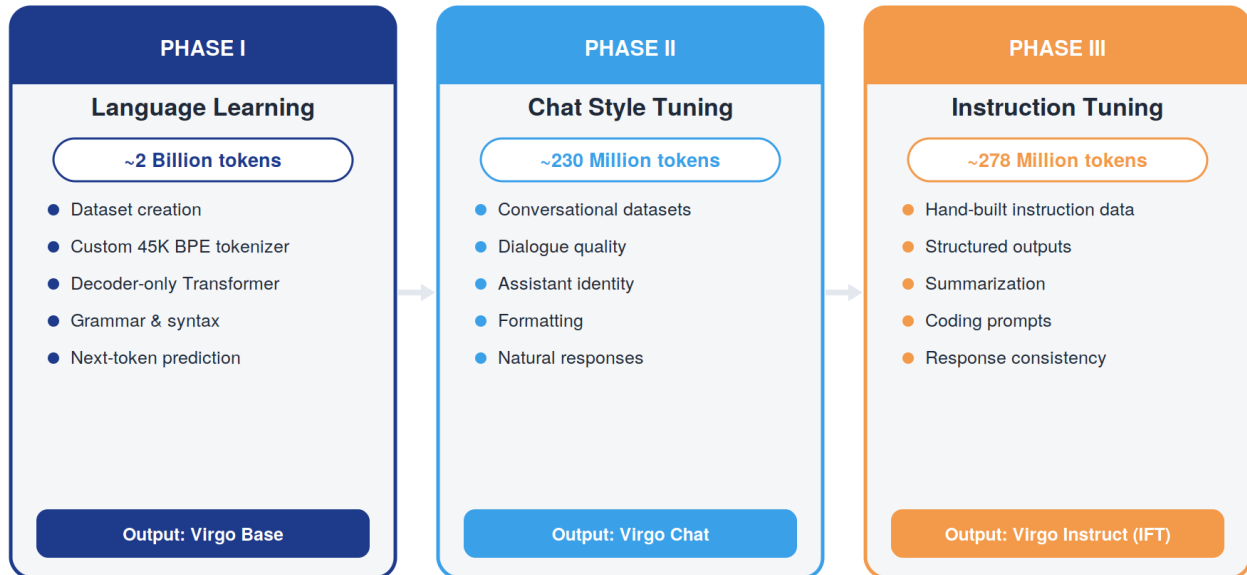


STUDENT REFLECTION

The jump from 'Neural Networks' to 'Transformers' was the hardest step in this entire chain. Everything before it had a fairly intuitive mechanical story. Self-attention did not — not until I re-implemented scaled dot-product attention by hand, on paper, with a toy 4-token example. That's the moment it actually clicked.

04 Virgo Development Roadmap

Virgo was developed across three deliberate phases, each with a distinct dataset, training objective, and evaluation criteria. This pretrain → chat-tune → instruction-tune structure mirrors the standard methodology used in modern language model development, and separating the phases made it possible to isolate the contribution of each stage to the final model's behavior.

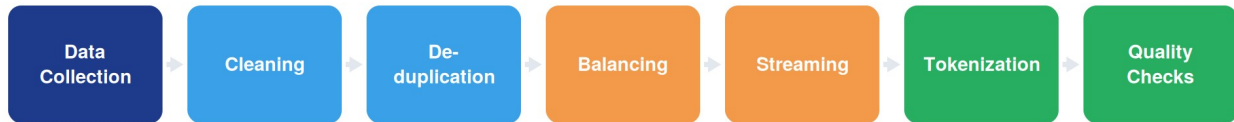


STUDENT REFLECTION

Phase III took the longest by far. Training compute for that stage was a fraction of Phase I, but the dataset behind it consumed months of manual work — writing prompts, generating and cleaning responses, balancing task categories so the model wouldn't overfit to any single instruction style.

05 Dataset Engineering

Model quality in language modeling is substantially determined by the quality of the underlying data pipeline. The Virgo pipeline moves raw text through six stages before it ever reaches the training loop, with each stage designed to remove a specific category of noise or bias from the corpus.



What each stage actually involved

- Collection: pulling from open web-text corpora, book-style writing, and public conversational datasets, kept within permissive licenses.
- Cleaning: stripping HTML remnants, boilerplate, broken Unicode, and duplicate whitespace that would otherwise inflate token counts.
- Deduplication: near-duplicate documents were removed after they were found to distort early loss curves by encouraging memorization over generalization.
- Balancing: ensuring no single domain (news, code, dialogue) dominated the mixture and skewed the model's output style.
- Streaming: chunking cleaned text into fixed-length sequences that could be streamed during training without loading the full corpus into memory.
- Quality checks: automated filters were supplemented with manual batch sampling, which consistently caught issues automation missed.

STUDENT REFLECTION

If there's one lesson Virgo taught me above all others, it's that a language model is only as good as the pipeline feeding it. I spent more hours cleaning and balancing text than I spent writing the model architecture itself.

06 Tokenizer

Virgo uses a custom Byte-Level BPE (Byte Pair Encoding) tokenizer with a 45,000-token vocabulary, trained from scratch on the pretraining corpus rather than reused from an existing model. Byte-level BPE was selected because it guarantees no out-of-vocabulary tokens; in the worst case, it falls back to raw byte encoding rather than failing on unseen input.



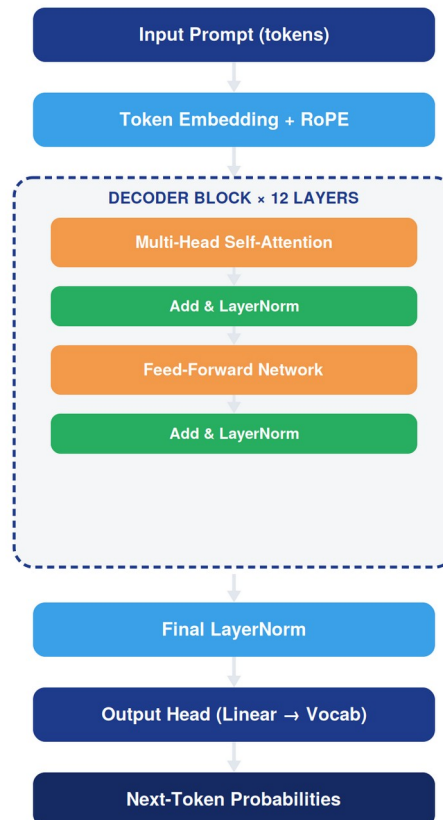
Training a dedicated tokenizer, rather than importing an off-the-shelf vocabulary such as GPT-2's, ensured the merge rules reflected the specific vocabulary and style distribution of the Virgo corpus.

STUDENT REFLECTION

That decision also meant debugging genuinely strange failures — like special chat-formatting tokens colliding with byte-pair merges — until I learned to reserve them explicitly outside the trainable vocabulary.

07 Transformer Architecture

Virgo is a decoder-only Transformer, architecturally aligned with the GPT family, comprising approximately 119.6 million parameters distributed across 12 layers, a 768-dimensional hidden size, and 12 attention heads per layer. Rotary Positional Embeddings (RoPE) were used in place of learned absolute position embeddings for their strong extrapolation properties on longer sequences.



Each of the 12 decoder blocks follows an identical structure: multi-head self-attention with a causal mask, followed by a residual Add & LayerNorm step, a position-wise feed-forward network, and a second Add & LayerNorm step. The residual connections proved architecturally critical for training stability at this depth.

STUDENT REFLECTION

Implementing RoPE myself, rather than using a library default, forced me to actually understand rotation matrices and relative position encoding at a level reading about them never had. And the residual connections weren't just theoretically important — my first version without properly scaled residuals refused to train past a few hundred steps.

Core specifications

Specification	Value
Parameters	≈ 119.6 Million
Layers	12 decoder blocks
Hidden size	768
Attention heads	12
Positional encoding	RoPE (Rotary)

Specification	Value
Tokenizer	Byte-Level BPE, 45K vocabulary
Context length	Fixed-length training sequences

08 Training Journey

Training Virgo Base on approximately 2 billion tokens using consumer-grade hardware required careful application of memory and compute optimization techniques: mixed-precision training (fp16/bf16) to fit larger batches into limited VRAM, gradient accumulation to simulate a larger effective batch size, and frequent checkpointing to guard against interrupted runs.

ENGINEERING NOTE

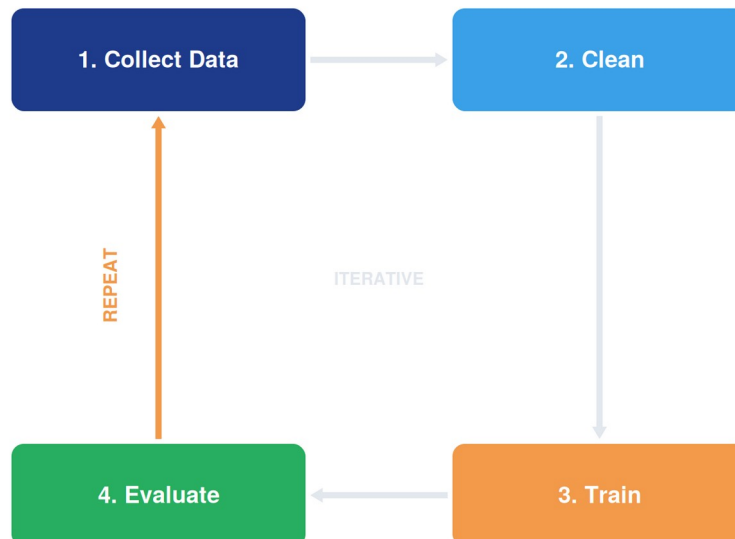
Learning-rate warmup was not optional. Early runs used a flat learning rate from step one, causing the loss to spike and fail to recover. A short linear warmup followed by cosine decay resolved this immediately.

STUDENT REFLECTION

Watching the training loss curve become one of the more meditative parts of this project. In the early steps, the model's outputs were pure noise — repeated characters, no word boundaries. Somewhere around a few hundred million tokens in, actual English words began appearing. That was a genuinely exciting moment to watch happen on my own machine.

09 The Development Loop

Beyond the three major phases, day-to-day progress on Virgo followed a smaller, tighter iteration cycle applied repeatedly within each phase: collect data, clean it, train on it, evaluate the resulting checkpoint, and use that evaluation to decide what to collect or fix next. This loop, rather than any single training run, is what actually produced measurable improvement over time.



Why the loop matters

Each iteration of the loop is intentionally short. Rather than committing to a single large training run, smaller cycles made it possible to catch data quality issues, tokenizer edge cases, and architectural instabilities early, before they were baked into a multi-day training run.

STUDENT REFLECTION

This loop is the real story behind Virgo, more than any individual architecture decision. Most of my 'progress' on any given week wasn't writing new code — it was going around this circle again: pull more data, notice it was messier than I thought, clean it, retrain, look at the outputs, and realize what to fix next. Respecting this cycle, instead of trying to skip straight to a final training run, is what actually got Virgo working.

10 Challenges & Lessons Learned

This chapter documents the significant technical failures encountered during development. Each is presented alongside its root cause and resolution, since these failures were, in aggregate, as instructive as any successful training run.

CHALLENGE: CUDA Out-of-Memory

An early architecture used a batch size that appeared reasonable on paper but ignored activation memory during backpropagation. Gradient checkpointing and gradient accumulation resolved this, at the cost of slower steps.

CHALLENGE: Silent Tokenizer Bugs

An early tokenizer version merged a newline character into unrelated word pairs due to a preprocessing bug, quietly corrupting a portion of every document. Diagnosing this required roughly a week of investigating generation quality before the root cause was located.

CHALLENGE: Checkpoint Corruption

Two separate training runs were lost to interrupted checkpoint writes following unexpected shutdowns. The fix: write checkpoints to a temporary file first, and rename them into place only once the write completes.

LEARNING: Dataset Bugs Hide as Model Bugs

More than once, hyperparameters were tuned for hours to fix a problem that was actually a mislabeled or duplicated data shard. The data is now checked before the architecture whenever a model behaves unexpectedly.

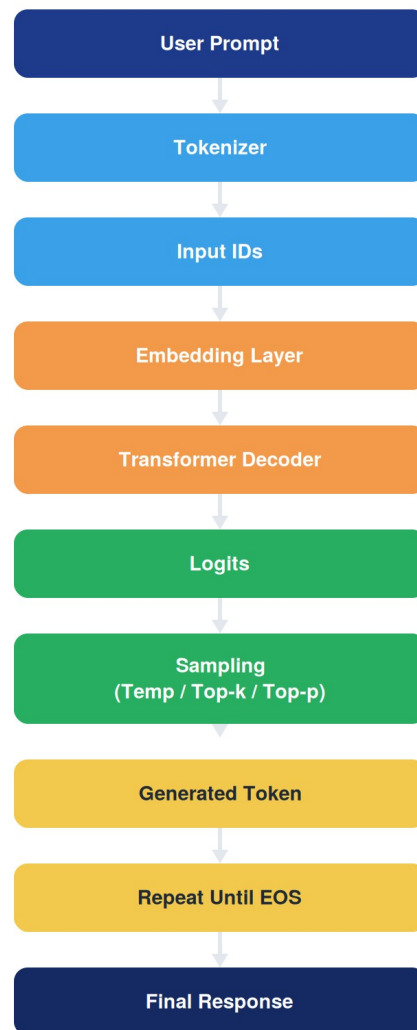


STUDENT REFLECTION

This is the chapter I almost didn't want to write, because it means admitting how many times Virgo simply broke. But it's also the most useful chapter for anyone attempting something similar.

11 Virgo Inference Cycle

Text generation in Virgo follows a fixed, repeating cycle executed once per output token. Tracing this loop end-to-end was essential for debugging generation quality and understanding where in the pipeline a given failure originated.



Decoding strategy

Virgo samples the next token using a combination of temperature scaling, top-k filtering, and top-p (nucleus) sampling, rather than deterministically selecting the highest-probability token. Temperature reshapes the probability distribution prior to sampling: lower values increase determinism and repetitiveness, while higher values increase diversity at the cost of coherence. Top-k restricts sampling to the k most likely tokens; top-p restricts it to the smallest token set whose cumulative probability exceeds a given threshold.

STUDENT REFLECTION

In practice, combining a moderate temperature with top-p sampling gave the most natural-sounding Virgo responses — tuning that ratio by trial and error taught me more about sampling theory than any single paper did.

12 Capability Assessment

The dashboard below reflects observed behavior from Virgo Instruct across a manual test set of prompts spanning writing, conversation, reasoning, and code — it is a qualitative internal assessment, not a formal benchmark score.



STUDENT REFLECTION

Being honest about what Virgo can and cannot do matters more to me than inflating its abilities. It currently follows a strict prompt → response interaction model, with no persistent memory, web search, tool use, image generation, or long-context support — deliberate scope boundaries for a V1.0 built by one person in one year, not oversights.

13 Current Limitations

Virgo should be evaluated as an early-stage 120-million-parameter Small Language Model. At this scale, the following limitations are expected rather than anomalous:

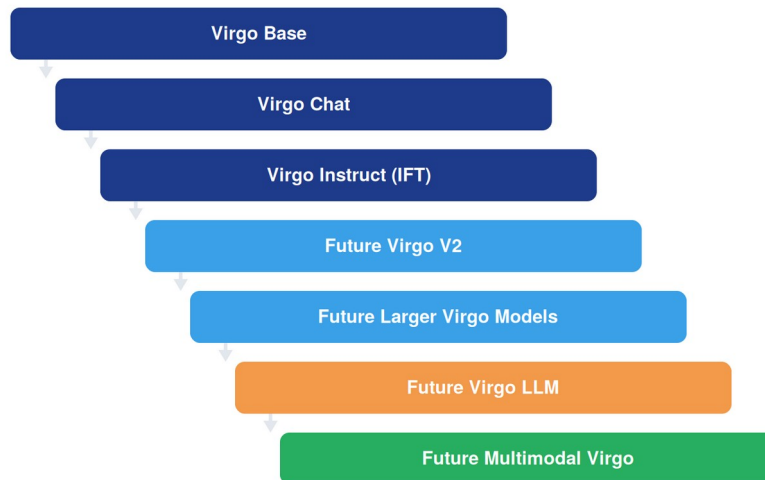
- Reasoning and multi-step instruction following remain the weakest capabilities and are an active area for future work.
- Programming ability is limited to short, simple snippets — Virgo struggles with debugging or multi-file logic.
- Mathematical reasoning beyond basic arithmetic is unreliable and should not be trusted.
- Hallucinations occur, particularly on factual or niche knowledge questions outside the training distribution.
- Long, multi-turn conversations degrade in coherence due to the model's limited context window.

STUDENT REFLECTION

None of this is discouraging to me — if anything, it's the clearest possible roadmap for Virgo V2. A model that already did everything perfectly would leave nothing left for me to learn from.

14 Future Vision

Virgo Instruct represents the first working generation of a planned model family rather than a terminal deliverable. Subsequent work is scoped around the limitations identified in the previous chapter.



Future development will prioritize stronger reasoning, improved instruction following through larger and more diverse datasets, extended context windows, retrieval-augmented generation, persistent memory, tool use, web search, and eventual image understanding and multimodal capability — while preserving training efficiency on non-data-center hardware.

STUDENT REFLECTION

Virgo Instruct is a milestone, not a destination. I think of it as the first working generation of a family of models I intend to keep building and scaling as I learn more.

15 Join the Virgo Community

Virgo is maintained as an open-source learning project. As a model built without institutional research infrastructure, it is expected to benefit substantially from external contribution, review, and testing. Contributions of any scope — from a single dataset fix to a training-efficiency improvement — are welcome via the standard workflow below.



Ways to contribute

- Dataset improvements, cleaning, and new instruction categories
- Documentation, tutorials, and educational explainers for beginners
- Model evaluation, benchmarking, and honest capability testing
- Bug fixes and training-efficiency optimizations
- New feature development and research ideas for Virgo V2

16 Final Author Message

When I started this project, I could not have told you what a residual connection was for. I definitely could not have told you what a CUDA out-of-memory error looked like at 2 a.m., or how it feels to lose a three-day training run to a single unhandled exception during checkpoint saving. Virgo taught me both, along with a hundred smaller lessons in between.

This project is not impressive because Virgo writes better than GPT-4 — it doesn't, and it was never meant to. It is meaningful to me because every single component in it, from the tokenizer's merge rules to the attention masks to the sampling logic, is something I understood well enough to build, break, and rebuild myself.

If you are a student reading this and thinking about starting something similar: you do not need the perfect roadmap, the perfect GPU, or the perfect dataset. You need a blank file, the willingness to watch a loss curve fail to converge a dozen times, and the patience to debug the thirteenth. Start before you feel ready. You will learn the rest on the way.

“Not Everybody Needs The Blueprint.”

— Punit Kumar Kashyap
Engineering Physics, IIT Dharwad